Canonical Even-Index Representations and Recursive Decomposition of Rectilinear Polygons

Representation, monotone hulls, and exact canonical verification

Wai-Shing Luk

physdes

October 24, 2026

Section 1

Outline

Outline

1. Outline
2. Introduction
3. Representation
4. Monotone Hulls
5. Recursive Decomposition
6. Verification
7. Conclusion

Section 2

Introduction

Rectilinear Geometry in Physical Design

- VLSI layouts are overwhelmingly **Manhattan**: every edge is horizontal or vertical.
- Rectilinear polygons model **obstacles**, **regions**, and **layout fracturing** patterns [1], [2].
- Coordinates are **integers**; rounding is not merely inaccurate but illegal.
- The polygons of interest are small: $n \leq 100$ vertices.
- Consequently, **quadratic algorithms are acceptable**; clarity and exactness dominate.

Problem Statement and Contributions

Input. A point sequence $S = (s_0, \dots, s_{m-1})$ with $s_i \in \mathbb{Z}^2$.

Tasks.

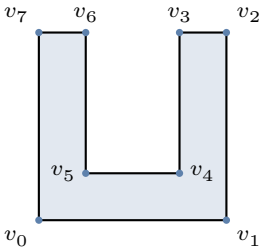
1. A canonical representation with an exact structural predicate.
2. Monotone and convex hulls that preserve the represented polygon.
3. Recursive decomposition into convex pieces and rectangles.
4. Cross-language verification (Python and C++20).

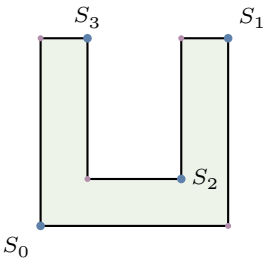
Contributions. A template-method treatment of x and y, a collinear-cleanup step for even-index hulls, and a canonicality invariant that area checks cannot replace.

Section 3

Representation

Full Boundary and Even-Index Encoding

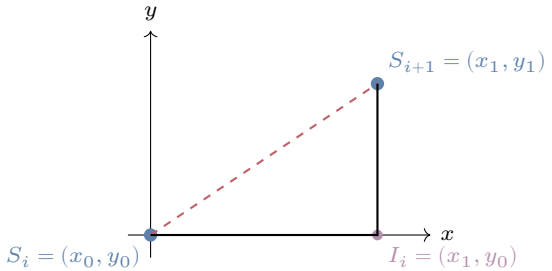




Definition. For a full boundary $V = (v_0, \dots, v_{n-1})$ with n even, the even-index encoding is $S = (v_0, v_2, \dots, v_{n-2})$.

- The odd-index vertices are implicit; only $m = n/2$ points are stored.
- The represented polygon is recovered by the x-first rule below.

x-First Expansion



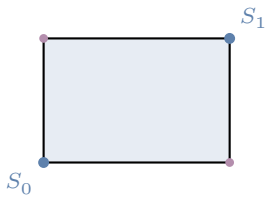
Definition (x-first expansion). For consecutive stored points $S_i = (x_0, y_0)$ and $S_{i+1} = (x_1, y_1)$, insert

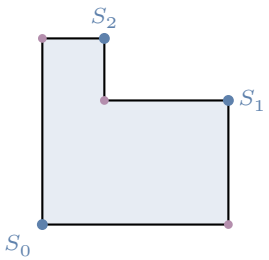
$$I_i = (x_{i+1}, y_i)$$

before S_{i+1}, including the closing pair $S_{m-1} \to S_0$.

- The diagonal chord $S_i S_{i+1}$ is **not** a boundary edge.
- The y-first form (x_0, y_1) reconstructs a different polygon.

Minimal Encodings





- **2 stored points** represent a rectangle: $E(S)$ has four corners.
- **3 stored points** represent a monotone / L-shaped piece: six corners.
- These are not degenerate cases; they are the base cases of the decomposition.

Lemma. $E(S)$ is a closed rectilinear polygon if and only if every cyclic consecutive pair of stored points differs in both coordinates.

Proof sketch. Each pair contributes a horizontal edge followed by a vertical edge; sharing an axis collapses one edge to zero. □

Canonical Form

Definition. A stored sequence $S = (s_0, \ldots, s_{m-1})$ is **canonical** if

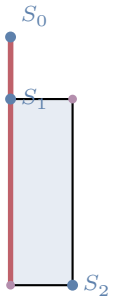
1. the s_i are pairwise distinct, and
2. for every i (indices modulo m),

$$x_i \neq x_{i+1} \quad \text{and} \quad y_i \neq y_{i+1}.$$

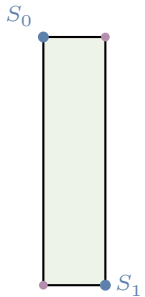
```
def is_canonical(S):
    if len(S) < 2:
        return True
    if len(set(S)) != len(S):
        return False
    return all(
        a[0] != b[0] and a[1] != b[1]
        for a, b in zip(S, S[1:] + S[:1])
    )
```

- Duplicate detection is $O(m^2)$ naively, or $O(m \log m)$ by sorting.
- The consecutive-pair test is $O(m)$.

Canonical Validation



not canonical



canonical

- $S = ((2,1),(2,0),(3,-3))$: the expansion contains two consecutive vertical segments and is **not** a canonical even-index list.
- Its canonical form is $((2,1),(3,-3))$.
- Area alone cannot detect this defect; the predicate is **structural**.

Orientation and Signed Area

For a simple polygon with vertices (x_i, y_i),

$$A = \frac{1}{2} \sum_i \left(x_i y_{i+1} - x_{i+1} y_i \right).$$

- $A > 0$: anticlockwise; $A < 0$: clockwise.
- The even-index encoding admits an $O(m)$ evaluation over the stored points.
- The sign drives the hull and cut branches and serves as a cheap postcondition.

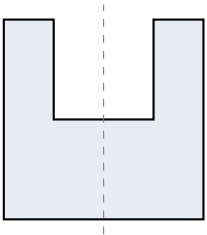
Lemma. The signed area is invariant under the x-first expansion E.

Proof sketch. Each inserted vertex is collinear with an axis-aligned edge and contributes zero net area. □

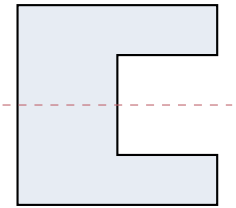
Section 4

Monotone Hulls

Monotonicity and Convexity



x-monotone, not y-monotone



y-monotone, not x-monotone

Definition. A rectilinear polygon is x-monotone if every vertical line meets it in a single interval; y-monotone is defined symmetrically.

Lemma. A rectilinear polygon is orthogonally convex if and only if it is both x-monotone and y-monotone.

Monotone Hull Algorithm

- Let p, c, n be the previous, current, and next stored vertices.
- A vertex is removed when monotonicity is violated and the turn has the wrong sign:

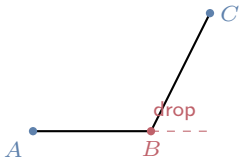
$$\Delta = (y_c - y_p)(x_n - x_c).$$

- A circular doubly-linked list makes removal $O(1)$.
- Two passes: minimum to maximum, then maximum to minimum.

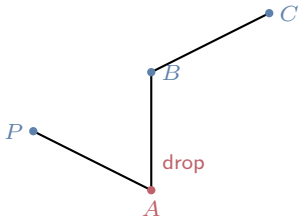
```
for c in chain:
    if dir(c) > dir(n) or dir(p) > dir(c):
        if wrong_sign(Delta):
            remove(c); c = prev(c)
        else:
            c = next(c)
    else:
        c = next(c)
```

Complexity. $O(m)$ time and $O(m)$ space per hull.

Collinear Cleanup



same y: drop the second



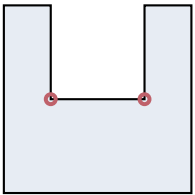
same x: drop the first

- A monotone hull can still contain collinear even-index points.
- Remove the second point of a same-y pair; remove the first of a same-x pair.
- Repeat until no adjacent pair shares an axis.

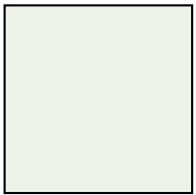
Lemma. The cleanup preserves the represented boundary and the signed area.

Proof sketch. The removed point lies in the interior of a straight boundary edge; deleting it merges two collinear edges into one. □

Hull Composition and Correctness



before



after

- x-hull then y-hull yields the rectilinear convex hull.
- Both hulls are idempotent on canonical inputs.
- The composition preserves the represented polygon and its area.
- With cleanup, the output satisfies `is_canonical`.

Section 5

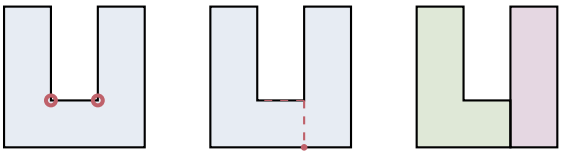
Recursive Decomposition

Three-Step Decomposition



- **Step 1: convex cut.** Remove concavities; simplify the remaining geometry.
- **Step 2: explicit 270°.** Resolve reflex corners that are stored vertices.
- **Step 3: implicit 270°.** Resolve reflex corners introduced by expansion.
- The final pieces are rectangles, suitable for layout fracturing and DRC [3], [4].

Cut Primitive



(a) concave vertices (b) minimum cut (c) split and recurse

- Find a concave vertex: a direction reversal with the correct area sign.
- Find the nearest vertex by Manhattan distance (horizontal or vertical).
- Insert the cut point; split the circular list; recurse on both pieces.

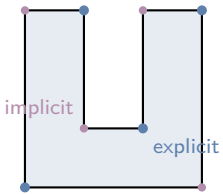
Base Cases and Recursion

Base cases.

- 2 stored points: a rectangle; stop.
- 3 stored points: a monotone / L-shaped piece; stop in the convex variant.
- The explicit variant has no 3-point base case; it continues until no concave vertex remains.

```
cut(v1):
    v2, v3 = v1.next, v1.next.next
    if v3 == v1:
        return [v1, v2]
    if triangle_base and v3.next == v1:
        return [v1, v2, v3]
    c = find_concave(v1)
    if c is None:
        return all_vertices(v1)
    vmin, vertical = find_min_dist(c)
    new = insert_cut(c, vmin, vertical)
```

Explicit and Implicit 270° Vertices



- **Explicit** $270°$**:** the reflex corner is a stored even-index vertex.
- **Implicit** $270°$**:** the reflex corner is created by the x-first expansion.
- The same compressed U-shape contains both kinds.
- The convex cut handles the shape; the explicit and implicit cuts handle the corner types.

Rectangle Cutting and Canonicality

- A rectangle piece has **2 stored points**: two opposite corners.
- The test `x_0 \ne x_1` and `y_0 \ne y_1` is exactly the canonical predicate for a 2-point list.
- Rectangle pieces are therefore canonical by construction.
- For larger pieces, canonicality must be checked explicitly; area and convexity do not imply it.

Correctness Invariants and Complexity

Invariants.

1. **Area:** the sum of signed piece areas equals the signed area of the input.
2. **Convexity:** every piece is rectilinearly convex.
3. **Canonicality:** every piece is a valid even-index list; where the algorithm guarantees it, `is_canonical` holds.

Complexity.

operation	cost
concave-vertex scan	$O(n)$
nearest-vertex scan	$O(n)$
insert / split	$O(1)$
total decomposition	$O(n^2)$

For $n \leq 100$, quadratic time is preferable to a complicated sub-quadratic implementation.

Section 6

Verification

Implementation and Tests

Python

- canonical predicate tests
- hull-before-check assertions
- area + canonical checks on cut pieces
- full suite: **627 passed**

C++20 (xmake)

- `xmake build test_recti`
- `xmake run test_recti`
- **139 test cases**
- **934 assertions**
- **0 failures**

- The hull is applied before the canonical check; raw creation is only an ordering, not a canonical representation.
- Three mixed even-index cut fixtures deliberately do not assert canonicality: their pieces can still contain implicit collinear points.

Limitations and Threats to Validity

1. **Representation-specific predicate.** `is_canonical` enforces the x-first even-index convention; a geometrically valid rectangle can be written with a same-axis pair and must be rewritten.
2. **Synthetic inputs.** The evaluation uses generated point sets, not foundry data; the claims are algorithmic, not signoff.
3. **Quadratic scaling.** Decomposition is $O(n^2)$; $n > 10^3$ is open.
4. **Cut completeness.** Some implicit collinear points require the cleanup phase; the cut algorithm alone does not guarantee canonical output.
5. **Robustness.** Integer arithmetic is exact; degenerate input handling is outside the present scope [5].

Section 7

Conclusion

Contributions and Open Problems

Contributions.

- A canonical even-index representation with an exact structural predicate.
- A template-method treatment of creation and monotone hulls.
- A collinear-cleanup lemma for even-index hulls.
- A recursive decomposition with explicit and implicit 270° handling.
- Cross-language verification in Python and C++20.

Open problems.

- Sub-quadratic decomposition for large polygons.
- Canonical output for every cut variant, without a post-processing pass.
- Integration with detailed routing and DRC flows.
- Three-dimensional rectilinear decomposition.

Q&A

- Code: `physdes-py` and `physdes-cpp` under the `luk036` organization.
- Thank you.

References I

[1] N. A. Sherwani, *Algorithms for VLSI physical design automation*, 3rd ed. Boston, MA: Kluwer Academic Publishers, 1999.

[2] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu, *VLSI physical design: From graph partitioning to timing closure*. Dordrecht: Springer, 2011.

[3] A. B. Kahng, C.-H. Park, X. Xu, and H. Yao, "Layout decomposition for double patterning lithography," in *Proc. IEEE/ACM int. Conf. Computer-aided design (ICCAD)*, 2008, pp. 465–472. doi: 10.1109/ICCAD.2008.4681616.

[4] L. W. Liebmann, "Layout impact of resolution enhancement techniques: Impediment or opportunity?" in *Proc. Int. Symp. Physical design (ISPD)*, 2003, pp. 110–117. doi: 10.1145/640000.640026.

[5] H. Edelsbrunner and E. P. Mücke, "Simulation of simplicity: A technique to cope with degenerate cases in geometric algorithms," *ACM Trans. Graph.*, vol. 9, no. 1, pp. 66–104, 1990.

References II